

Major Assignment - 1,2

Data Miners
Kavuri Goutham Chandra
Atharva Bodakhe
Arsh Jain
Raunit Kumar
Tushar Jain

November 26, 2025

Problem 1.1

1) Derivation of the Semi-Parametric Kernel $\tilde{K}$

Objective: Convert the semi-parametric model $y = p^\top \phi(z) \cdot x + b$ into a standard non-parametric form $y = \tilde{p}^\top \psi(x, z)$ by defining a new feature map $\psi(x, z)$ and a corresponding kernel $\tilde{K}$.

The original semi-parametric regression model is given by:

$$y = p^\top \phi(z) \cdot x + b, \quad (1)$$

where $p \in \mathcal{H}$ is the weight vector in the Hilbert space corresponding to $\phi(z)$, and $b \in \mathbb{R}$ is a scalar bias.

We rewrite equation (1) as an inner product of two augmented vectors. By distributing the scalar x , we get:

$$y = (p^\top \phi(z))x + b = \langle p, x\phi(z) \rangle_{\mathcal{H}} + b \cdot 1.$$

We can represent this sum as a single dot product in an augmented Hilbert space $\tilde{\mathcal{H}} = \mathcal{H} \oplus \mathbb{R}$. We define the new parameter vector $\tilde{p}$ and the new feature map $\psi(x, z)$ as follows:

$$\tilde{p} = \begin{bmatrix} p \\ b \end{bmatrix}, \quad \psi(x, z) = \begin{bmatrix} x \cdot \phi(z) \\ 1 \end{bmatrix}.$$

Verifying the inner product:

$$\tilde{p}^\top \psi(x, z) = \begin{bmatrix} p^\top & b \end{bmatrix} \begin{bmatrix} x\phi(z) \\ 1 \end{bmatrix} = p^\top (x\phi(z)) + b \cdot 1 = p^\top \phi(z) \cdot x + b.$$

This confirms that the new feature map ψ correctly encapsulates the semi-parametric model.

Now, we derive the kernel $\tilde{K}$ associated with ψ . By definition, the kernel value between two points $((x_1, z_1), (x_2, z_2))$ is the inner product of their feature maps:

$$\begin{aligned}
\tilde{K}((x_1, z_1), (x_2, z_2)) &= \langle \psi(x_1, z_1), \psi(x_2, z_2) \rangle \\
&= \begin{bmatrix} x_1 \phi(z_1) \\ 1 \end{bmatrix}^\top \begin{bmatrix} x_2 \phi(z_2) \\ 1 \end{bmatrix} \\
&= (x_1 \phi(z_1))^\top (x_2 \phi(z_2)) + 1 \cdot 1 \\
&= x_1 x_2 (\phi(z_1)^\top \phi(z_2)) + 1.
\end{aligned}$$

We are given that the kernel for the z -component is the polynomial kernel, $K_z(z_1, z_2) = \phi(z_1)^\top \phi(z_2) = (z_1^\top z_2 + c)^d$. Substituting this into the equation above, we obtain the final form of the kernel:

$$\boxed{\tilde{K}((x_1, z_1), (x_2, z_2)) = x_1 x_2 (z_1^\top z_2 + c)^d + 1}$$

2) Hyperparameter Tuning Results

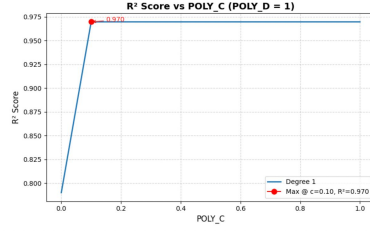
Methodology: We performed a grid search to determine the optimal hyperparameters for the polynomial kernel: the degree d and the coefficient c (coef0). We evaluated the model's performance using the R^2 score on the public test set for degrees $d \in \{1, 2, 3, 4\}$ and coefficients $c \in [0.1, 1.0]$.

Results: The validation R^2 scores for various combinations of d and c are summarized in Table 1.

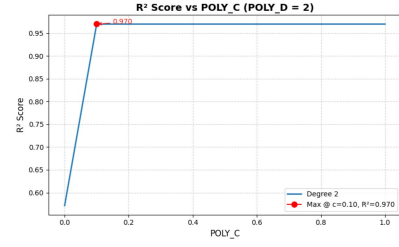
Table 1: R^2 Scores for different values of Degree (d) and Coef0 (c)

c (Coef0)	d=1	d=2	d=3	d=4
0.1	0.9697	0.96993	0.96995	0.96993
0.2	0.9697	0.96993	0.96994	0.96981
0.3	0.9697	0.96993	0.96994	0.96979
0.4	0.9697	0.96993	0.96994	0.96978
0.5	0.9697	0.96993	0.96994	0.96978
0.6	0.9697	0.96993	0.96994	0.96978
0.7	0.9697	0.96993	0.96994	0.96978
0.8	0.9697	0.96993	0.96994	0.96978
0.9	0.9697	0.96993	0.96994	0.96978
1.0	0.9697	0.96993	0.96994	0.96978

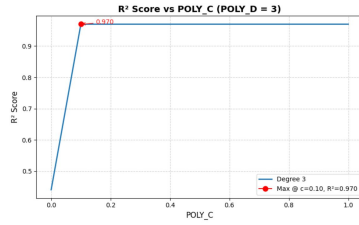
Visual Analysis: The plots below illustrate the R^2 score trends for degrees $d = 2, 3, 4$ as the coefficient c varies.



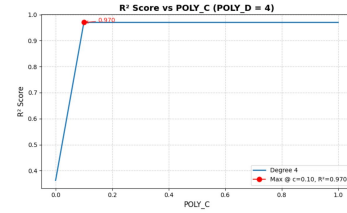
(a) Degree $d = 1$



(b) Degree $d = 2$



(c) Degree $d = 3$



(d) Degree $d = 4$

Figure 1: R^2 Score vs. Coef0 (c) for different polynomial degrees.

Conclusion: Based on the grid search, the model achieves the highest validation score of **0.96995** with the hyperparameters:

$$\boxed{\text{Degree } d = 3, \quad \text{Coef0 } c = 0.1}$$

But, time for kernel formation and training is also a factor we are taking $\mathbf{d=2}$ and $\mathbf{c=0.1}$ as our optimal hyperparameters.

Using the optimal hyperparameters ($d = 2, c = 0.1$), the model was evaluated on the secret dataset. The observed performance metrics are summarized below:

Table 2: Evaluation Metrics on Secret Dataset

Metric	Observed Value
Kernel Computation Time (t_{kernel})	0.3743 s
Training Time (t_{train})	1.4081 s
Test R^2 Score	0.9698

Problem 1.2

4)

Problem Description

Find non-negative delay vectors

$$x^{(0)}, x^{(1)} \in \mathbb{R}_{\geq 0}^{128}$$

(each containing 128 delays, for a total of 256 delays) such that the linear model produced by the XOR of the two arbiter PUFs matches the given vector

$$w \in \mathbb{R}^{1089}.$$

Each arbiter PUF has 32 stages, indexed by $0 \leq i \leq 31$. At every stage there are four delay parameters:

$$a_i, b_i, c_i, d_i.$$

Thus the delay vector for the first arbiter PUF is

$$x^{(0)} = (a_0, b_0, c_0, d_0, a_1, b_1, c_1, d_1, \dots, a_{31}, b_{31}, c_{31}, d_{31})^T \in \mathbb{R}^{128}.$$

Similarly, the 128-dimensional delay vector for the second arbiter PUF is

$$x^{(1)} = (p_0, q_0, r_0, s_0, p_1, q_1, r_1, s_1, \dots, p_{31}, q_{31}, r_{31}, s_{31})^T \in \mathbb{R}^{128}.$$

Method to find non-negative delays

Since the XOR–arbiter PUF model satisfies

$$\mathbf{w} = \mathbf{u} \otimes \mathbf{v},$$

where $\mathbf{u}$ and $\mathbf{v}$ are the model weight vectors for the two arbiter PUFs, each belonging to $\mathbb{R}^{33}$, our first goal is to recover a valid pair $(\mathbf{u}, \mathbf{v})$ that satisfies this relationship.

Then our next step is to recover the delay vectors $x^{(0)}$ and $x^{(1)}$.

Because $1089 = 33^2$, the given vector $w \in \mathbb{R}^{1089}$ is reshaped into a 33×33 matrix: The structure of this matrix is

$$w^T = \begin{bmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_{32} \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_{32} \\ \vdots & \vdots & \ddots & \vdots \\ u_{32} v_1 & u_{32} v_2 & \cdots & u_{32} v_{32} \end{bmatrix}.$$

Flattening this matrix column-by-column produces the 1089-dimensional vector w , which is the given XOR–PUF linear model.

Finding u & v

The XOR-arbiter PUF model satisfies

$$W = uv^\top,$$

where $u, v \in \mathbb{R}^{33}$ are the linear model vectors corresponding to the two individual arbiter PUFs. Since W is the outer product of two vectors, it has the explicit form

$$W = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_{33} \end{bmatrix} [v_1 \ v_2 \ \cdots \ v_{33}],$$

so every row of W is a scalar multiple of the vector $v^\top$. Therefore,

$$\boxed{\text{rank}(W) = 1} \tag{*}$$

Taking the singular value decomposition

$$W = U\Sigma V^\top,$$

we can obtain a dominant singular value σ_1 and singular vectors u_1 and v_1 , while the remaining singular values $\sigma_2, \sigma_3, \dots$ are small and correspond only to noise because of (*). Thus

$$\boxed{W \approx \sigma_1 u_1 v_1^\top}.$$

Scaling u & v

1. From the leading singular triplet (σ_1, u_1, v_1) of W , we form

$$u = \sqrt{\sigma_1} u_1, \quad v = \sqrt{\sigma_1} v_1.$$

2. The model is scale-ambiguous, since for any nonzero c ,

$$(cu) \left(\frac{1}{c}v \right)^\top = uv^\top.$$

3. To remove this ambiguity, we pin one coordinate of u to a fixed value (we did it to 1) and rescale u and v accordingly.
4. Without scale fixing, u and v can drift to very large or tiny magnitudes, making the recovered delays unstable; pinning ensures a consistent scale and numerically stable delay recovery.

Framing the equations to get the delays

We must recover the two delay vectors $x^{(0)}, x^{(1)}$ (one per arbiter) by solving

$$\boxed{\begin{aligned} Ax^{(0)} &= u, \\ Ax^{(1)} &= v, \end{aligned}}$$

where

$$u, v \in \mathbb{R}^{33}, \quad x^{(0)}, x^{(1)} \in \mathbb{R}^{128}, \quad A \in \mathbb{R}^{33 \times 128}.$$

The unknown vector for each arbiter is ordered stage-by-stage as

$$x = (a_0, b_0, c_0, d_0, a_1, b_1, c_1, d_1, \dots, a_{31}, b_{31}, c_{31}, d_{31})^\top.$$

From

$$\alpha_i = \frac{1}{2}(a_i - b_i + c_i - d_i), \quad \beta_i = \frac{1}{2}(a_i - b_i - c_i + d_i),$$

and

$$u_0 = \alpha_0, \quad u_i = \alpha_i + \beta_{i-1} \quad (1 \leq i \leq 31), \quad u_{32} = \beta_{31},$$

we get the compact block form given below

$$A = \begin{bmatrix} B_0 & 0 & 0 & \cdots & 0 \\ B_1 & B_0 & 0 & \cdots & 0 \\ 0 & B_1 & B_0 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & \cdots & 0 & B_1 & B_0 \end{bmatrix}, \quad B_0 = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2} \end{bmatrix}, \quad B_1 = \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} \end{bmatrix},$$

Solving for the delays by inverting the equations

To invert the arbiter PUF linear model, we use the simplified assumption

$$\beta_i = 0, \quad i = 0, \dots, 30.$$

This is valid because many different delay configurations generate the same model vector u ; we only need *one* non-negative solution, not the physical delays.

Under this assumption,

$$u_i = \alpha_i \quad (i = 0, \dots, 30),$$

and from the definition of α_i ,

$$\alpha_i = \frac{1}{2}(a_i - b_i + c_i - d_i),$$

we may absorb the factor $\frac{1}{2}$ into the final scale and treat each stage as satisfying the two difference equations

$$a_i - b_i = u_i, \quad c_i - d_i = u_i,$$

with the constraints $a_i, b_i, c_i, d_i \geq 0$.

Each stage is therefore solved independently:

- If $u_i \geq 0$, choose

$$a_i = u_i, \quad b_i = 0, \quad c_i = u_i, \quad d_i = 0,$$

which satisfies both differences and keeps all delays non-negative.

- If $u_i < 0$, choose

$$a_i = 0, \quad b_i = -u_i, \quad c_i = 0, \quad d_i = -u_i,$$

which again enforces non-negativity and the required differences.

This produces a valid non-negative delay assignment for every stage that reconstructs the given arbiter model u .

For the last stage, the model relations are

$$u_{31} = \alpha_{31} + \beta_{30}, \quad u_{32} = \beta_{31}.$$

Since we set all earlier $\beta_i = 0$, we have

$$\alpha_{31} = u_{31}, \quad \beta_{31} = u_{32}.$$

Using the definitions

$$\alpha_{31} = \frac{a_{31} - b_{31} + c_{31} - d_{31}}{2}, \quad \beta_{31} = \frac{a_{31} - b_{31} - c_{31} + d_{31}}{2},$$

add and subtract to obtain

$$\alpha_{31} + \beta_{31} = a_{31} - b_{31}, \quad \alpha_{31} - \beta_{31} = c_{31} - d_{31}.$$

Substituting $\alpha_{31} = u_{31}$ and $\beta_{31} = u_{32}$ gives

$$a_{31} - b_{31} = u_{31} + u_{32}, \quad c_{31} - d_{31} = u_{31} - u_{32}.$$

These are solved under non-negativity exactly as in the earlier stages.